Logistic Regression from Scratch & CIFAR-10 Classification

Bruno Fernandez

March 2026

(a) Write a Python function that computes the maximum likelihood estimator for $\beta = [\beta_1, \beta_2]^T$ using the Newton-Raphson algorithm (see Chapter 4.4 of Hastie et al. for a detailed description). You may use the skeleton of this function that is given in `A2_logistic_regression_simulation.py`. All required matrix algebra can be done with the NumPy library.

Recall that the log-likelihood of a binary classification model is given by

$$\ell(\beta) = \sum_{i=1}^{n} y_i \beta^T \mathbf{x}_i - \log(1 + e^{\beta^T \mathbf{x}_i})$$

where $\mathbf{x}_i^T$ denote the features of observation i, and are the rows of the matrix $\mathbf{X}$. The gradient of the log-likelihood is

$$\nabla_\beta \ell(\beta) = \sum_{i=1}^{n} \mathbf{x}_i \left(y_i - \frac{1}{1 + e^{-\beta^T \mathbf{x}_i}} \right) = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \dots & \mathbf{x}_n \end{bmatrix} \begin{bmatrix} y_1 - \pi_1 \\ y_2 - \pi_2 \\ \vdots \\ y_n - \pi_n \end{bmatrix} = 0 \iff \mathbf{X}^T(\bar{y} - \bar{\pi})$$

where we set $\pi_i = \frac{1}{1+e^{-\beta^T \mathbf{x}_i}}$. Furthermore, the Hessian of $\ell(\beta)$ with respect to β is

$$\nabla_\beta^2 \ell(\beta) = -\mathbf{X}^T \mathbf{W} \mathbf{X}$$

where

$$\mathbf{W} = \begin{bmatrix} \pi_1(1-\pi_1) & 0 & \dots & 0 \\ 0 & \pi_2(1-\pi_2) & \dots & 0 \\ \vdots & \vdots & \ddots & 0 \\ 0 & 0 & \dots & \pi_n(1-\pi_n) \end{bmatrix}$$

giving the Newton-Raphson update formula

$$\beta_{\text{new}} = \beta_{\text{old}} - (\nabla_\beta^2 \ell(\beta))^{-1} \nabla_\beta \ell(\beta) = \beta_{\text{old}} + (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T (\bar{y} - \bar{\pi})$$

Furthermore, we add a suitable termination criterion. In the code, we chose $\|\nabla_\beta \ell(\beta)\| < \varepsilon$ with a very low tolerance parameter $\varepsilon = 10^{-12}$, as we can expect

fast (second-order) convergence from the Newton-Rhapson method. In other words, we terminate once the function is almost stationary in the neighbourhood of β_{old}.

See code in the appendix.

(b) Compute the mean of the MLE for both parameters over $S = 1000$ simulations. Also plot a histogram for the MLE of both parameters. Repeat the Monte Carlo experiment with a sample size of $n = 100$ instead of $n = 1000$. Discuss your findings.

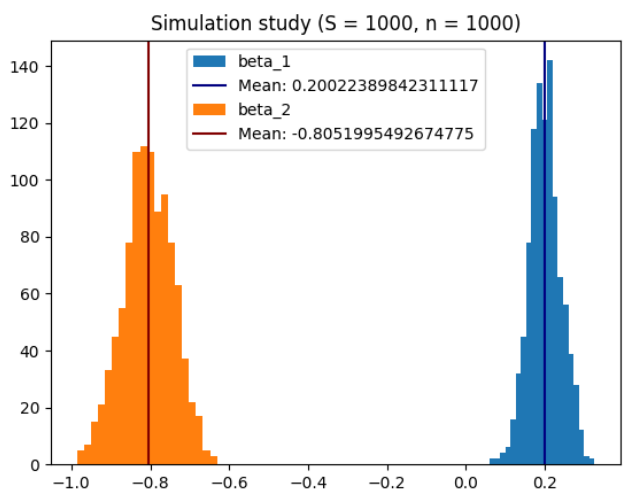

Figure 1: Estimates for β_1 and β_2 computed over $n = 1000$ replications.

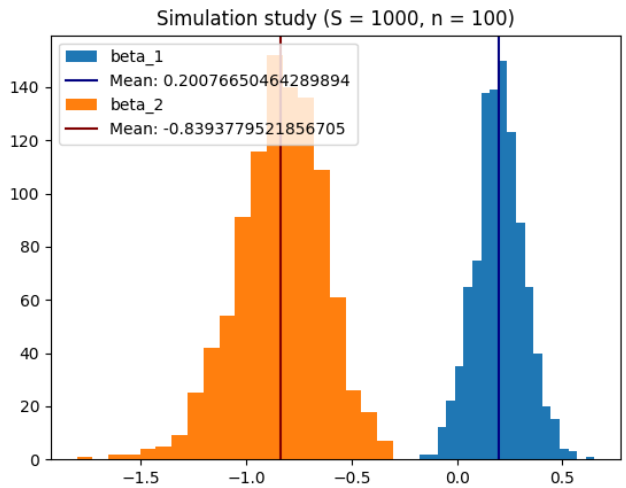

Figure 2: Estimates for β_1 and β_2 computed over $n = 100$ replications.

Figure 1 depicts the results for $n = 1000$ and figure 2 depicts the results for $n = 100$. We observe that both sample sizes are capable of very accurately estimating $\beta_1 = 0.2$. However, for the smaller sample size, the histogram for β_2 displays a heavier tail towards lower values for β_2, which reflects in a slightly lower mean estimate compared to the true value of β_2.

The skewness arises because of how the sample data is distributed. Samples are concentrated around $\mu = (-0.5, 1)$ and $\mu = (2, -1)$. This means that for x_1 and x_2, there is a 'gap' in the probability density in between the means for the respective coordinates, where little to no information can be extracted. For a coordinate with relatively large $|\beta^*|$, there is a substantial risk of one of these half-distributions being almost completely devoid of one type of label. In that case, the model falsely assumes that the logistic function is very steep with respect to x_2, yielding a high estimate for β_2. When sample sizes increase, this simulation result occurs less frequently, reducing it's effect of inflating the mean estimate for β_2.

(c) Run the Newton-Raphson algorithm that you have written in part (a). Python will display an error message due to a matrix singularity. Compute the rank of the matrix of features X, which is called `x_train` in the python script. Explain what is going wrong.

We have that the MLE must necessarily satisfy

$$\nabla \ell_\beta(\beta) = 0 \iff X^T(\bar{y} - \bar{\pi}) = 0$$

The rank of '`x_train`' and its transpose is $100 = n$. If $\text{rank}(X^T) = n$, then

the necessary condition only holds when $\bar{y} - \bar{\pi} = 0 \iff \bar{\pi} = \bar{y}$. However, since $y_i \in \{0, 1\}$ and the codomain of $\bar{\pi}$ is $(0, 1)$ (for finite β-values), this equality can never hold, and the necessary condition for the MLE can thus not be satisfied.

(d) In order to solve the issue arising in (c), we add a ridge penalty to the log-likelihood criterion:

$$J(\beta) = -\ell(\beta) + \lambda \sum_{j=1}^{p} \beta_j^2, \tag{1}$$

where we take $\lambda = 1$ and $\ell(\beta)$ denotes the log-likelihood function of the logistic regression model. Adapt the Newton-Raphson algorithm such that we obtain the regularised estimator of β that minimizes $J(\beta)$. Evaluate the predictive performance of this estimator using the function `prediction_accuracy`. Discuss your findings.

We have that the gradient of the new objective function is

$$\nabla_\beta J(\beta) = -\nabla_\beta \ell(\beta) + 2\lambda\beta = -\mathbf{X}^T(\bar{y} - \bar{\pi}) + 2\lambda\beta$$

and the Hessian is

$$\nabla_\beta^2 J(\beta) = \mathbf{X}^T\mathbf{W}\mathbf{X} + 2\lambda\mathbf{I}$$

giving the Newton-Raphson update formula

$$\beta_{\text{new}} = \beta_{\text{old}} - (\nabla_\beta^2 J(\beta))^{-1}\nabla_\beta J(\beta) = \beta_{\text{old}} + (\mathbf{X}^T\mathbf{W}\mathbf{X} + 2\lambda\mathbf{I})^{-1}(\mathbf{X}^T(\bar{y} - \bar{\pi}) - 2\lambda\beta)$$

The ridge penalty effectively biases β-coefficients toward smaller values and ensures that the objective function has a non-singular Hessian (solving the issue in c)). Running the modified function results in an estimator with `prediction_accuracy` of `0.9948269001193792`, given the termination criteria specified in 1a. In other words, with the new estimated parameters β the correct label is predicted in 99.5% of cases. Given the relatively small size of the training data (100 images), this is a rather impressive result!

1 Question 2 - CIFAR-10 Image Classification

In this question we apply regularized logistic regression to the CIFAR-10 dataset, which consists of 60,000 color images (50,000 training, and 10.000 test) each belonging to one of the following 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck. Each image is 32x32 pixels with three color channels (red, green and blue), giving $p = 3,072$ features per observation. For the experiments, we have used *sklearn*, *numpy* and *matplotlib.pyplot* libraries.

(a) - Data loading and Pre-processing

We download the dataset directly from the *University of Toronto* and we loaded using the pickle format. We concatenate five training batches to produce the full training matrix $\mathbf{X_{train}}$, which has shape (50,000x3,072), and a label vector $\mathbf{y_{train}}$ of length 50,000. The test set has shape (10,000x3,072).

Moreover, as pixels have integer values between 0 and 255 (0 corresponds to black, and 255 is white), we normalize them to the range [0,1] by dividing the data $\mathbf{X}$ by 255. This is necessary because some optimization solvers require the data to be normalized to guarantee fast convergence.

As required, we provide the image number 320 of the dataset, which is a horse, and its label. Also, we show the dimensions of the training and testing data. The code of this part can be found in (2.2).

```
The dimensions of the matrix X_train is:  (50000, 3072) . The dtype is:  uint8
The dimensions of the vector y_train is:  (50000,) . The dtype is:  int64
The dimensions of the matrix X_test is:  (10000, 3072) . The dtype is:  uint8
The dimensions of the matrix y_test is:  (10000,) . The dtype is:  int64
```

Figure 3: Dimensions of the data set (training and testing).

Figure 4: One representative image from the CIFAR-10 dataset with its label.

(b) - Logistic Regression with a Fixed Regularization Parameter.

We fit a multinomial logistic regression model using different values of the regularization parameter C, which is the inverse of the ridge parameter λ defined in the course. We use $C = \{1, 0.1, 0.01\}$, which corresponds to the values $\lambda = 1/C = \{1, 10, 100\}$. We use *max-iter* $= 1000$ to guarantee that the optimization solver (in our case 'lbfgs') converges to the solution. The model is fitted on the full normalized training set. The results for the three values are: As observed, the test accuracy is approximately 40%, which reflects poor effi-

```
Training accuracy : 0.4829  (48.3%)
Test accuracy     : 0.3802  (38.0%)
```

Figure 5: Training and test accuracy for $C = 1$

```
Training accuracy : 0.4734  (47.3%)
Test accuracy     : 0.4042  (40.4%)
```

Figure 6: Training and test accuracy for $C = 0.1$

```
Training accuracy : 0.4516  (45.2%)
Test accuracy     : 0.4182  (41.8%)
```

Figure 7: Training and test accuracy for $C = 0.01$

ciency in the classification of the Logistic Regression model. However, we notice that as we reduce C (i.e. we increase the parameter λ), the training accuracy decreases while the test one increases.

(c) - Cross-Validation with accuracy score.

Now, we use the *LogisticRegressionCV* with $K = 4$ stratified folds to select the regularization parameter C automatically. Here, stratified folds are used to guarantee that each fold maintains the same percentage of samples for each class as the original dataset. We search over 12 different values from a log-space grid taking values from $C = 10^{-4}$ to $C = 10$

The cross-validated accuracy is averaged across all folds and classes. Next, we show the resulting log-log plot:

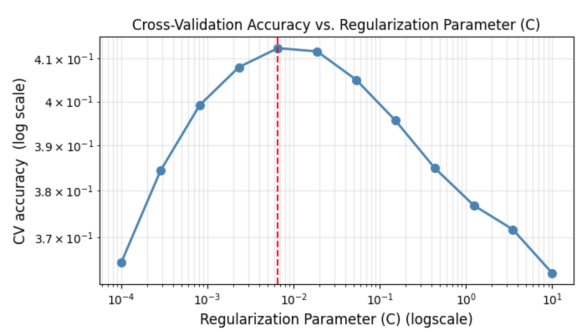

Figure 8: Cross-validated accuracy as a function of the regularization parameter C. The red-dashed line indicates the best value.

The curve shows that for small values of the regularization parameter C (i.e. high values of λ, meaning strong regularization) the model starts to suffer from underfitting. However, for large values of C (weak regularization), the model begins to overfit. The optimal value is $C^* = 0.00658$, which corresponds to a penalty of $\lambda^* = 1/C^* \approx 152$.

```
LogisticRegressionCV
LogisticRegressionCV(Cs=array([1.00000000e-04, 2.84803587e-04, 8.11130831e-04, 2.31012970e-03,
       6.57933225e-03, 1.87381742e-02, 5.33669923e-02, 1.51991108e-01,
       4.32876128e-01, 1.23284674e+00, 3.51119173e+00, 1.00000000e+01]),
                     cv=4, max_iter=1000, n_jobs=-1)
```

Figure 9: Logistic regression model with CV.

```
Best C: 0.00658
```

Figure 10: Optimal value.

(d) - Cross-Validation with logarithmic score rule.

Now, we will use a logarithm scoring rule (in particular, *log-loss*), which is a transformation of the likelihood function. The main difference with the accuracy metric is that log-loss takes into account the uncertainty of predictions by penalizing models more heavily for confidently incorrect predictions. For example, a model which says that $\mathbb{P}(\text{horse}) = 0.91$ when the true label is 'horse' will score better than one with $\mathbb{P}(\text{horse}) = 0.5$, despite both being correctly classified by the accuracy metric.

We use cross-validation with predicted probabilities (*cross_val_predict* with *method='predict_proba'*) to estimate the probability for each training sample using models that did not see that sample during training. As in (c) we used *StratifiedKFold* method. Then, we calculate the log loss (*metrics.log_loss*) based on these estimates. This process is repeated for each value of the regularization parameter C using the same grid as in part (c).

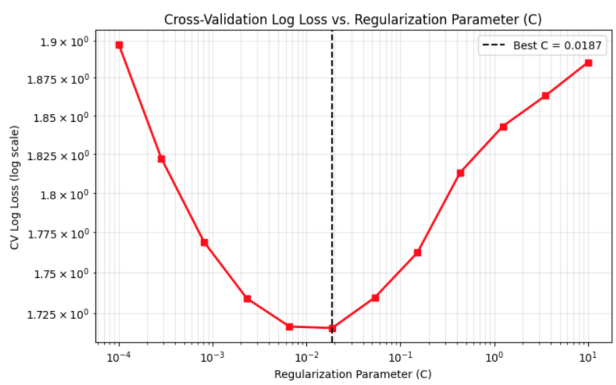

Figure 11: Cross-validation log-loss as a function of C. The black dashed line marks the optimal value.

```
\Optimal C_value based on Log Loss: 0.01874
```

Figure 12: Optimal regularization parameter C.

It can be observed that the optimal value obtained with the log-loss score is $C^* = 0.01874$. This value is different from the one found in part (c), which was $C^* = 0.00658$. Note that both criteria have a valley shaped curve with an optimum in the middle. However, log-loss selects a larger parameter C (smaller λ), meaning less regularization.

This disagreement can be explained from the fact that the accuracy score only considers if the predictions are correct, without taking into account the model's confidence, while the log-loss score evaluates the predicted probabilities and reflects the model's confidence.

(e) - Final model: Selection of C and error reporting.

Since the values obtained from (c) and (d) are different, we evaluate three candidates: $C1 = 0.00658$ (best by accuracy), $C2 = 0.01874$ (best by log-loss), and $C3 = (0.00658 + 0.01874)/2 = 0.01266$ (midpoint). Note that all three values remain in the flat region near the optimum for both curves.

We choose $C2 = 0.01874$ as the primary choice because the log-loss score because it encourages models to be not only accurate but also confident in their predictions.

```
--- Results for Optimal C = 0.00658 ---
Train Accuracy: 0.4458
Train Log Loss: 1.6487
Test Accuracy:  0.4164
Test Log Loss:  1.7026
```

Figure 13: Training and test accuracy/log-loss for $C1 = 0.00658$.

```
--- Results for Optimal C = 0.01874 ---
Train Accuracy: 0.4580
Train Log Loss: 1.6141
Test Accuracy:  0.4177
Test Log Loss:  1.7047
```

Figure 14: Training and test accuracy/log-loss for $C2 = 0.01874$.

```
--- Results for Optimal C = 0.01266 ---
Train Accuracy: 0.4539
Train Log Loss: 1.6265
Test Accuracy:  0.4182
Test Log Loss:  1.7022
```

Figure 15: Training and test accuracy/log-loss for $C3 = 0.01266$.

We can observe that for all three values of C and for both metrics, the training error is lower than the test one. This is expected since the model was optimized in the training data set.

The three candidates perform very similarly, which is consistent as the three values remain in the flat part of both CV curves, where there is the optima.

If we observe the plots from (c) and (d), we see that the CV accuracy and log-loss for $C1 = 0.00658$ are approximately 0.412 and 1.72, respectively. For the parameter $C2 = 0.01874$, we deduce that the CV accuracy and log-loss

are approximately 0.411 and 1.717, respectively. These cross-validation scores are slightly different from the ones obtained from the ones obtained training the model with the full dataset. However, qualitatively, $C2$ is the one with the lower log-loss score, as we observed for the full dataset. Then, our cross-validation process provided a reliable and fair estimate of how the model performs on new data.

As a curiosity, analyzing the test accuracy and the test log-loss, we can deduce that for the test set the parameter that has lower log-loss score and higher accuracy is the middle point $C3 = 0.01266$.

2 Appendix

This appendix is divided into two sub-appendices, one per question. Appendix A is aimed for Question 1, and appendix B for Question 2.

2.1 Appendix A - Code for Question 1

The sections below correspond to parts (a)-(d) of Question 1.

Part (a) - Code for Newton-Raphson MLE estimation.

```
def logistic_regression_NR(features, target, num_steps, tolerance):
    #initialization of beta
    beta = np.zeros(features.shape[1])
    # Features is x matrix
    # Target is y matrix


    for step in range(num_steps):
        signal = np.dot(features, beta)
        p = logistic(signal)
        gradient = features.T @ (target - p)
        if np.linalg.norm(gradient) < tolerance:
            return beta, step

        #compute gradient and hessian

        W = np.diag(p * (1 - p))
        beta += np.linalg.inv(features.T @ W @ features) @ features
    .T @ (target - p)

    return beta, -1

simulated_labels = logistic_simulation(simulated_features,
    beta_star)

beta, step = logistic_regression_NR(simulated_features,
    simulated_labels, 100, 1e-12)
print(f'Estimated beta: {beta}')
```

Part (b) - Code for Simulation Study.

```
S=1000
MLE_results = np.zeros((S, 2))
for i in range(S):

    #generate labels y for every simulation
    simulated_labels = logistic_simulation(simulated_features,
    beta_star)
    #compute the MLE for every simulation
    MLE, step = logistic_regression_NR(simulated_features,
    simulated_labels, 100, 1e-12)
    MLE_results[i] = MLE

beta_1 = np.mean(MLE_results[:,0])
beta_2 = np.mean(MLE_results[:,1])
```

```
print(f'Beta 1: {beta_1}')
print(f'Beta 2: {beta_2}')
#compute the means of estimated parameters beta_1 and beta_2
#make a histogram for the MLE of beta_1 and beta_2

plt.figure()
plt.hist(MLE_results[:,0],bins=20, label='beta_1')
plt.axvline(beta_1, color=(0.0, 0.0, 0.5, 1.0), label=(f'Mean: {beta_1}'))
plt.hist(MLE_results[:,1],bins=20, label='beta_2')
plt.axvline(beta_2, color=(0.5, 0.0, 0.0, 1.0), label=(f'Mean: {beta_2}'))
plt.title(f'Simulation study (S = {S}, n = {n})')
plt.legend()
plt.show()
```

Part (d) - Code for Ridge Regression.

```
# Regularisation parameter
lambda_0=1

def logistic_regression_NR_penalized(features, target, num_steps, tolerance, lambda_0):
    beta = np.zeros(features.shape[1])
    for step in range(num_steps):
        signal = np.dot(features, beta)
        p = logistic(signal)
        # modified gradient
        gradient = - features.T @ (target - p) + 2 * lambda_0 * beta
        if np.linalg.norm(gradient) < tolerance:
            return beta, step

        W = np.diag(p * (1 - p))
        beta += np.linalg.inv(features.T @ W @ features + 2 * lambda_0 * np.diag(np.ones_like(beta))) @ (features.T @ (target - p) - 2 * lambda_0 * beta)
    return beta, -1

tol = 1e-12
beta, step = logistic_regression_NR_penalized(x_train, y_train, 100, tol, lambda_0)

y_predicted = logistic_forecast(x_test, beta)

print(f'Accuracy: {prediction_accuracy(y_predicted, y_test)}')
print(f'Steps to reach gradient norm tolerance threshold {tol}: {step}')
```

2.2 Appendix B - Code for Question 2

The sections below correspond to parts (a)-(e) of Question 2.

Part (a) - Data loading and pre-processing

```
!wget https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
```

```
!tar -xvzf cifar-10-python.tar.gz

import numpy as np
import matplotlib.pyplot as plt
from sklearn import linear_model, model_selection, metrics
import warnings
warnings.filterwarnings('ignore')


def unpickle(file):
    import pickle
    with open(file, 'rb') as fo:
        dict = pickle.load(fo, encoding='latin1')
    return dict

dict1 = unpickle("./cifar-10-batches-py/data_batch_1")
dict2 = unpickle("./cifar-10-batches-py/data_batch_2")
dict3 = unpickle("./cifar-10-batches-py/data_batch_3")
dict4 = unpickle("./cifar-10-batches-py/data_batch_4")
dict5 = unpickle("./cifar-10-batches-py/data_batch_5")
test = unpickle("./cifar-10-batches-py/test_batch")
meta_data = unpickle("./cifar-10-batches-py/batches.meta")
label_names = meta_data["label_names"]


X_train = np.concatenate((dict1["data"],dict2["data"],dict3["data"
    ],dict4["data"],dict5["data"]))
y_train = np.concatenate((dict1["labels"],dict2["labels"],dict3["
    labels"],dict4["labels"],dict5["labels"]))
X_test = test["data"]
y_test = test["labels"]

def data_to_image(x):
    return(x.reshape(3,32,32).transpose(1,2,0))

def plot_image(image, title=""):
    fig = plt.imshow(data_to_image(image))
    plt.title(title)
    fig.axes.set_axis_off()
    plt.show()

# as a verification that everything is working correctly, plot an
    image
image_nr = 320
plot_image(X_train[image_nr,:],label_names[y_train[image_nr]])

#We transform vectors y_train and y_test to arrays to compute its
    dimensions

y_test = np.array(y_test)
y_train = np.array(y_train)

print("The dimensions of the matrix X_train is: ", X_train.shape, "
    . The dtype is: ", X_train.dtype )
print("The dimensions of the vector y_train is: ", y_train.shape, "
    . The dtype is: ", y_train.dtype )
```

```python
print("The dimensions of the matrix X_test is: ", X_test.shape, ". The dtype is: ", X_test.dtype )
print("The dimensions of the matrix y_test is: ", y_test.shape, ". The dtype is: ", y_test.dtype )

#Now, in order to use logistic functions and other functions we will normalize the data. In X_train, for example, we have 50000 different images (n), and 3072 rows which corresponds to red, green and blue channel values (each one consisting of 1024 cells of the matrix). Then, each cell consist of a pixel value between 0 and 255. Then when scaling the data, we will divide by 255 to obtain values between 0 and 1.

X_train_normalized = X_train/255.0
X_test_normalized = X_test/255.0
```

Part (b) - Fixed regularization parameters We try with three different values for C.

```python
from sklearn.linear_model import LogisticRegression
model1 = LogisticRegression(C = 1, max_iter= 1000, solver='lbfgs')
model1.fit(X_train_normalized, y_train)

acc_train = model1.score(X_train_normalized, y_train)
acc_test = model1.score(X_test_normalized, y_test)

print(f"  Training accuracy : {acc_train:.4f}  ({acc_train*100:.1f}%)")
print(f"  Test accuracy     : {acc_test:.4f}  ({acc_test*100:.1f}%)")


model2 = LogisticRegression(C = 0.1, max_iter= 1000, solver='lbfgs')
model2.fit(X_train_normalized, y_train)

acc_train = model2.score(X_train_normalized, y_train)
acc_test = model2.score(X_test_normalized, y_test)

print(f"  Training accuracy : {acc_train:.4f}  ({acc_train*100:.1f}%)")
print(f"  Test accuracy     : {acc_test:.4f}  ({acc_test*100:.1f}%)")


model3 = LogisticRegression(C = 0.01, max_iter= 1000, solver='lbfgs')
model3.fit(X_train_normalized, y_train)

acc_train = model3.score(X_train_normalized, y_train)
acc_test = model3.score(X_test_normalized, y_test)

print(f"  Training accuracy : {acc_train:.4f}  ({acc_train*100:.1f}%)")
print(f"  Test accuracy     : {acc_test:.4f}  ({acc_test*100:.1f}%)")
```

Part (c) - Cross Validation with Accuracy Score.

```
from sklearn.linear_model
importLogisticRegressionCV

#we select 12 values for C in the range [1e-4,100]
C_values = np.logspace(-4,1,12)

#in order to do cross-validation with K = 4 folds, we set cv = 4
model2 = LogisticRegressionCV(Cs = C_values, solver = 'lbfgs',
    max_iter = 1000, cv = 4, n_jobs = -1)
model2.fit(X_train_normalized, y_train)

#Now, to find the scores for the different values of C_values, we
    need to use one of the attributes that return the function
    LogisticRegressionCV. In particular, we use the scores_ one,
    which returns a dict with classes as the keys, and the values
    as the grid of scores obtained during cross-validating each
    fold. In particular, each value is an array of shape (n_folds,
    n_Cs).

scores_model2_raw = np.array(list(model2.scores_.values())) #
    returns an array of dimensions (10,4,12), i.e. (number of
    classes, number of folds, number of C_values)
scores_model2 = scores_model2_raw.mean(axis = 0).mean(axis = 0) #
    for each class, we do the mean for the different number of
    folds and the different number of classes. The result is an
    array with dimensions (number of C_values)

best_C_value = model2.C_[0] #returns the value of C that maps to
    the best score, repeated n_classes times, then, we only select
    one, for example the one corresponding to the first class.
print(f"  Best C: {best_C_value:.5f}")

plt.figure(figsize = (7,4))
plt.loglog(C_values, scores_model2, 'o-', color='steelblue',
    linewidth=2, markersize=7)
plt.axvline(best_C_value, color='red', linestyle='--', linewidth
    =1.5,
            label=f'Best C = {best_C_value:.4f}')
plt.xlabel('Regularization Parameter (C) (logscale)', fontsize=12)
plt.ylabel('CV accuracy  (log scale)', fontsize=12)
plt.title('Cross-Validation Accuracy vs. Regularization Parameter (
    C)')
plt.grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.show()
```

Part (d) - Cross Validation with Log-loss.

```
from sklearn.model_selection import StratifiedKFold,
    cross_val_predict
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss

'''
With KFold, some folds may have less samples of some classes,
    making the cross validation noisy (introducing bias).
```

```python

However, StratifiedKFold guarantees us that each fold has the same class proportions as the original dataset
'''

kf = StratifiedKFold(n_splits = 4, shuffle = True)

C_values = np.logspace(-4,1,12)

log_losses = []

for c in C_values:
  model = LogisticRegression(C = c, max_iter = 1000, solver = 'lbfgs')
  y_pred_probab = cross_val_predict(model, X_train_normalized, y_train, cv = kf, method = 'predict_proba', n_jobs = -1)

  loss = log_loss(y_train, y_pred_probab)
  log_losses.append(loss)

best_C_value_logloss = C_values[np.argmin(log_losses)]
print(f"\Optimal C_value based on Log Loss: {best_C_value_logloss:.5f}")

plt.figure(figsize = (8,5))
plt.plot(C_values, log_losses, marker='s', color='red', linewidth=2)
plt.axvline(best_C_value_logloss, color='black', linestyle='--', linewidth=1.5,
            label=f'Best C = {best_C_value_logloss:.4f}')
plt.xscale('log')
plt.yscale('log')
plt.xlabel('Regularization Parameter (C)')
plt.ylabel('CV Log Loss (log scale)')
plt.title('Cross-Validation Log Loss vs. Regularization Parameter (C)')
plt.legend()
plt.grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.show()
```

Part (e) - Final models. We have tried 3 different values for C. One found in (c), the other found in (d), and finally, the mean between these two values.

```python
from sklearn.metrics import accuracy_score, log_loss
from sklearn.linear_model import LogisticRegression

train_accuracies = []
train_loglosses = []
test_accuracies = []
test_loglosses = []
optimal_Cs = []

optimal_C1 = 0.00658 #the one obtained from (c)

final_model1 = LogisticRegression(C = optimal_C1, max_iter = 1000, solver = 'lbfgs', n_jobs = -1)
```

```python

final_model1.fit(X_train_normalized, y_train)


#now we compute predictions and probabilities for test and train sets

train_pred = final_model1.predict(X_train_normalized)
train_proba = final_model1.predict_proba(X_train_normalized)

test_pred = final_model1.predict(X_test_normalized)
test_proba = final_model1.predict_proba(X_test_normalized)

train_accuracies.append(accuracy_score(y_train, train_pred))
train_loglosses.append(log_loss(y_train, train_proba))
test_accuracies.append(accuracy_score(y_test, test_pred))
test_loglosses.append(log_loss(y_test, test_proba))
optimal_Cs.append(optimal_C1)

print(f"--- Results for Optimal C = {optimal_C1} ---")
print(f"Train Accuracy: {train_accuracies[-1]:.4f}")
print(f"Train Log Loss: {train_loglosses[-1]:.4f}")
print(f"Test Accuracy: {test_accuracies[-1]:.4f}")
print(f"Test Log Loss: {test_loglosses[-1]:.4f}")


optimal_C2 = 0.01874 #the one obtained from (d)

final_model2 = LogisticRegression(C = optimal_C2, max_iter = 1000, solver = 'lbfgs', n_jobs = -1)

final_model2.fit(X_train_normalized, y_train)


#now we compute predictions and probabilities for test and train sets

train_pred = final_model2.predict(X_train_normalized)
train_proba = final_model2.predict_proba(X_train_normalized)

test_pred = final_model2.predict(X_test_normalized)
test_proba = final_model2.predict_proba(X_test_normalized)

train_accuracies.append(accuracy_score(y_train, train_pred))
train_loglosses.append(log_loss(y_train, train_proba))
test_accuracies.append(accuracy_score(y_test, test_pred))
test_loglosses.append(log_loss(y_test, test_proba))
optimal_Cs.append(optimal_C2)

print(f"--- Results for Optimal C = {optimal_C2} ---")
print(f"Train Accuracy: {train_accuracies[-1]:.4f}")
print(f"Train Log Loss: {train_loglosses[-1]:.4f}")
print(f"Test Accuracy: {test_accuracies[-1]:.4f}")
print(f"Test Log Loss: {test_loglosses[-1]:.4f}")


optimal_C3 = (0.01874+0.00658)/2 #the one obtained from (d)
```

```python

final_model3 = LogisticRegression(C = optimal_C3, max_iter = 1000, solver = 'lbfgs', n_jobs = -1)

final_model3.fit(X_train_normalized, y_train)


#now we compute predictions and probabilities for test and train sets

train_pred = final_model3.predict(X_train_normalized)
train_proba = final_model3.predict_proba(X_train_normalized)

test_pred = final_model3.predict(X_test_normalized)
test_proba = final_model3.predict_proba(X_test_normalized)

train_accuracies.append(accuracy_score(y_train, train_pred))
train_loglosses.append(log_loss(y_train, train_proba))
test_accuracies.append(accuracy_score(y_test, test_pred))
test_loglosses.append(log_loss(y_test, test_proba))
optimal_Cs.append(optimal_C3)

print(f"--- Results for Optimal C = {optimal_C3} ---")
print(f"Train Accuracy: {train_accuracies[-1]:.4f}")
print(f"Train Log Loss: {train_loglosses[-1]:.4f}")
print(f"Test Accuracy: {test_accuracies[-1]:.4f}")
print(f"Test Log Loss: {test_loglosses[-1]:.4f}")
```